

Guide d'étude : manipuler des données avec le tidyverse

Ce qu'il faut savoir faire pour les oraux et l'évaluation sur table

Ce que ce module ajoute

La semaine dernière, vous avez appris les structures de R (vecteurs, types, `data.frame`) et leurs pièges silencieux : `NA` dans une condition, casse des catégories, `sort()` au lieu de `order()`, recyclage. Ce module apprend à **manipuler** un tableau avec le *tidyverse* : filtrer, résumer par groupe, changer de forme, joindre deux tableaux.

Le *tidyverse* rend le code plus **lisible**, donc plus facile à vérifier, mais il ne supprime aucun des pièges de la semaine dernière. Il en ajoute deux, propres à la manipulation de tableaux : les **jointures** qui dupliquent ou perdent des lignes sans prévenir, et le **groupement** qui persiste après `summarise()`. C'est sur ces erreurs, qu'aucun message ne signale, que portent les questions.

Comment ce module est évalué

Comme le reste du cours, **en présence et sans outil d'IA** :

- à l'**évaluation sur table** (50 % de la note du cours) : [À PRÉCISER : sur papier ou en salle informatique surveillée, durée, documents permis] ;
- lors des **oraux** individuels (25 % chacun) : [À PRÉCISER : quel oral porte sur le tidyverse].

On ne vous demandera pas d'écrire un long programme, ni de connaître par cœur le nom de chaque argument. On vérifiera que vous **comprenez** ce que fait une chaîne de commandes : prédire le nombre de lignes d'un résultat, expliquer pourquoi un résumé est faux, anticiper l'effet d'une modification. Ce sont les formats des pauses « À vous » des diapos, des sections « Et si... ? » des TP et de l'exercice formatif.

Les types de questions

Type	Ce qu'on vous demande	Exemple déjà vu
Prédire	Donner le résultat (souvent : le nombre de lignes) sans exécuter	<code>filter(mesures, signal > 1)</code> avec un <code>NA</code> (exercice formatif, Q2)

Type	Ce qu'on vous demande	Exemple déjà vu
Erreur	Expliquer pourquoi un résultat est faux, ou un message d'erreur	Un filtre « 2019 ou 2020 » mal écrit garde toutes les lignes (TP1, « Et si... ? »)
Et si	Anticiper l'effet d'une modification	<code>inner_join()</code> au lieu de <code>left_join()</code> (TP3)
Expliquer	Justifier un concept avec vos mots	Pourquoi une jointure à gauche peut ajouter des lignes (diapos)
Écrire	Produire une chaîne courte	Moyenne de <code>E2</code> par année (TP2, Q1)

À l'oral, chaque réponse est suivie d'une **relance** : « pourquoi ? », « comment l'auriez-vous vérifié ? ». Une réponse juste sans explication ne suffit pas.

Ce que vous devez savoir faire

T-IMP : importer et vérifier

- Choisir entre `read_csv()` et `read_tsv()` selon le séparateur.
- Lire le compte rendu d'importation (lignes, colonnes, type de chaque colonne).
- Expliquer pourquoi une colonne numérique peut être lue comme du texte (`<chr>`) : virgule décimale, `n/a`, unité dans la cellule ; utiliser l'argument `na`.
- Vérifier une importation avec `glimpse()`, `dim()` ou `spec()`.

Où le travailler : diapos « Lire un fichier » et « Vérifier l'importation », TP1 (Q0), TP2 (Q6).

T-PIP : le *pipe*

- Réécrire une imbrication de fonctions avec `|>`, et l'inverse.
- Lire une chaîne de commandes à voix haute (« puis... puis... »).
- Savoir que `%>%` fait la même chose.

Où le travailler : diapos « Le pipe » et pause « À vous » associée, exercice formatif Q1.

T-VRB : les verbes de `dplyr`

- Utiliser `select()`, `filter()`, `mutate()`, `arrange()`, `count()`, `slice_max()`.
- Distinguer `select()` (renvoie un tableau) de `pull()` (renvoie un vecteur, comme `$`).
- Nettoyer une colonne de texte avant d'analyser : `str_to_lower()`, `str_trim()`, et retirer les lignes en double avec `distinct()` ; repérer ces problèmes avec `count()`.

- Traduire une opération de R de base en *tidyverse* et inversement (diapo « Traduire le script de la semaine dernière »).
- Savoir qu'un verbe renvoie un **nouveau** tableau : sans assignation (`<-`), rien n'est modifié.
- Savoir qu'une colonne créée avec un nom existant **remplace** l'ancienne.

Où le travailler : section 4 des diapos, TP1.

T-FIL : filtrer sans se tromper

- Combiner des conditions avec `,` ou `&` (et), `|` (ou), `%in%` (l'une des valeurs).
- Expliquer pourquoi `filter(x == 1 | 2)` garde toutes les lignes, et pourquoi `filter(x == c("a", "b"))` en garde environ la moitié (recyclage).
- Prévoir qu'une condition `NA` écarte la ligne, et vérifier que les parties font le tout.
- Savoir que `arrange()` place toujours les `NA` à la fin.

Où le travailler : diapos « `filter()` et les `NA` » et pause « À vous : prédire le résultat », TP1 (« Et si... ? »), exercice formatif Q2 et Q3.

T-GRP : résumer par groupe

- Utiliser `group_by()` + `summarise()`, ou `summarise(..., .by = )`.
- Distinguer `summarise()` (une ligne par groupe) de `mutate()` (autant de lignes qu'avant).
- Distinguer `n()` (nombre de lignes) de `sum(n)` (somme d'une colonne) et de `n_distinct()` ; se demander **ce que représente une ligne**.
- Expliquer le **groupement résiduel** après un `summarise()` à plusieurs groupes, son effet sur un `sum()` ultérieur, et comment l'éviter (`.groups = "drop"`, `ungroup()`, `.by`, `count()`).

Où le travailler : section 5 des diapos, TP2 (« Et si... ? » 1, 4 et 5), exercice formatif Q4 et Q5.

T-PIV : changer la forme d'un tableau

- Reconnaître un tableau *tidy* (une variable par colonne, une observation par ligne, une valeur par case), et dire **ce qui cloche** dans un tableau qui ne l'est pas : deux variables dans une colonne, deux valeurs dans une case, des valeurs en noms de colonnes, une observation répartie sur deux tableaux.
- Passer du format large au format long (`pivot_longer()`) et inversement (`pivot_wider()`).
- Prévoir les dimensions du résultat (lignes × colonnes pivotées).
- Expliquer l'erreur obtenue quand on pivote ensemble du texte et des nombres.

Où le travailler : section 6 des diapos (quiz « ce tableau est-il *tidy* ? »), TP2 (Q3 à Q5), exercice formatif Q6.

T-JOI : joindre deux tableaux

- Choisir entre `left_join()`, `inner_join()`, `full_join()`, `semi_join()` et `anti_join()`.
- Préciser la clé avec `by`, y compris quand elle porte un nom différent (`join_by(a == b)`).
- Prévoir le nombre de lignes d'une jointure quand une clé est **en double** ou **sans correspondance**.

- Vérifier une jointure : lignes avant et après, `count(cle) |> filter(n > 1)`, `anti_join()`, argument `relationship`.

Où le travailler : section 7 des diapos, TP3, exercice formatif Q7.

T-AUD : vérifier une analyse

- Repérer dans un script `tidyverse` les erreurs **silencieuses** : clé en double, clé mal saisie, groupement résiduel, `NA` écartés par un filtre, `count()` qui compte des lignes et non des individus.
- Distinguer ce qui est faux de ce qui est correct (ne pas « corriger » un `na.rm = TRUE` justifié).
- Expliquer pourquoi ces erreurs viennent souvent **des données** et ne se repèrent qu'en vérifiant les données elles-mêmes.

Où le travailler : diapos « Nettoyer avant d'analyser », « Vérifier une jointure », « Attention : le groupement persiste », et la traduction du script de la semaine dernière.

Comment vous préparer

1. **Refaites sans ordinateur** les pauses « À vous », les « Et si... ? » des TP et l'exercice formatif : écrivez votre prédiction, *puis* vérifiez dans RStudio.
2. Pour chaque chaîne de commandes, prenez l'habitude de prédire le **nombre de lignes** du résultat : c'est la vérification la plus rapide, et la plus souvent demandée.
3. Quand une prédiction est fausse, **cherchez pourquoi** avant de passer à la suite : c'est ce qu'on vous demandera d'expliquer à l'oral.
4. Reprenez la traduction du script de la semaine dernière **seul**, puis vérifiez que vous retrouvez les mêmes résultats.
5. Les solutionnaires des TP donnent les réponses et les explications : lisez surtout les explications.

